The AI Contact Layer: Bridging Intelligent Behavior and the Physical and Digital World

Wang Luqiang
ai-lib

March 2026

Preprint Version: 1.5

Abstract

This paper introduces the AI Contact Layer, the second in a five-paper series on AI Native Architecture. While Paper 1 addresses the execution runtime mechanics, this paper addresses the conceptual question: what is the nature of the interface between AI intelligence and the external world?

The Contact Layer mediates between intelligent behavior and external capabilities, enabling agents to access models, tools, services, data sources, and devices through structured capability interfaces. We present the theoretical foundation distinguishing intelligence from action, demonstrate how the contact layer bridges cognition and execution, and position it within the broader AI Native Architecture.

We argue that the Intelligence-Action separation is not merely an architectural convenience, but a fundamental design principle for AI-native systems. This work provides the conceptual framework that unifies execution infrastructure with the deeper philosophical question of how AI systems interact with the real world.

Keywords: AI Contact Layer, Intelligence-Action Interface, AI Native Architecture, Capability Mediation, Action Architecture, Control Plane, Capability Routing

1 Introduction

1.1 The Limitation of Intelligence Alone

Artificial intelligence systems have achieved remarkable progress in reasoning, language understanding, and multimodal perception. Large-scale models from providers such as OpenAI [11], Anthropic [12], Google, and DeepSeek demonstrate increasingly sophisticated cognitive capabilities. However, a critical limitation remains:

Intelligence alone does not produce real-world impact.

For an AI system to perform meaningful tasks, it must interact with external systems. Examples include:

- Retrieving information from knowledge bases
- Invoking APIs to perform actions
- Controlling devices in the physical world
- Executing automation workflows
- Interacting with digital services

In other words, **AI must connect its internal reasoning with external action**. This connection is what we define as the AI Contact Layer.

1.2 The Missing Conceptual Layer

While execution layers and runtime systems address implementation concerns, there exists a deeper conceptual question: What is the nature of the interface between AI intelligence and the external world?

This question has both philosophical and practical dimensions:

- **Philosophical:** How do we understand the boundary between thought and action in AI systems?
- **Practical:** How do we architect systems that reliably bridge reasoning and execution?

The Contact Layer concept addresses both dimensions, providing:

1. A conceptual framework for understanding AI-world interaction
2. An architectural model for implementing this interaction
3. A unifying perspective that connects execution infrastructure with broader system design

1.3 Relationship to the Execution Layer Paper

This paper builds upon the AI Execution Layer architecture described in our companion work [1]. The relationship between these works can be understood through the lens of the AI Action Layer:

Action Layer Decomposition: The Action Layer comprises two subcomponents:

- **Contact Layer:** Routing, policy evaluation, provider selection (the "control plane")
- **Execution Runtime:** API invocation, protocol translation, error handling (the "data plane")

Complementary Focus: The Execution Layer paper describes the *runtime component*—how capabilities are invoked, translated, and normalized. This paper describes the *contact component*—how routing decisions are made, policies evaluated, and providers selected.

Together, they form the complete Action Layer: one handles the mechanics of execution, the other handles the logic of routing.

Table 1: Comparison of Execution Layer and Contact Layer Papers

Aspect	**Execution Layer Paper [1]**	**Contact Layer Paper (This Work)**
Focus	Runtime mechanics	Conceptual framework
Question	How to execute?	Why mediate?
Component	Execution Runtime	Contact Layer
Layer	Data plane	Control plane
Key Constraint	Minimality	Intelligence-Action separation

1.4 Contributions

The primary contributions are as follows:

1. **Conceptual Framework:** We define the Contact Layer as the interface between AI intelligence and the external world, distinguishing it from both cognitive processes and execution mechanisms.
2. **Theoretical Foundation:** We establish the Intelligence-Action distinction as a fundamental principle of AI-native system design.

3. **Architectural Integration:** We demonstrate how the Contact Layer relates to execution runtimes and orchestration systems, clarifying terminology boundaries.
4. **Practical Implications:** We show how this conceptual framework informs the design of capability-mediated AI systems.

2 Intelligence vs. Action: A Fundamental Distinction

2.1 Two Fundamental Aspects

To understand the role of the contact layer, we must first distinguish two fundamental aspects of AI systems:

Intelligence (Cognition): Internal cognitive processes including:

- Reasoning about goals and constraints
- Planning sequences of actions
- Understanding language and context
- Making decisions
- Memory and learning

These processes occur entirely within the AI model or agent. Intelligence represents the system's ability to *think*.

Action: Interactions with the environment including:

- Calling an API
- Retrieving data from a database
- Sending a message
- Controlling a device
- Triggering automation

These processes involve interaction with external systems. Action represents the system's ability to *do*.

2.2 Why Separation Matters

Without action, intelligence remains isolated. An AI system might reason perfectly about a task but lack the ability to execute it. Consider:

- A planning system that generates perfect plans but cannot invoke tools
- A reasoning engine that identifies optimal solutions but cannot implement them
- A decision maker that chooses correctly but cannot act on decisions

The separation of intelligence and action is not merely architectural—it is fundamental. AI systems require a structured interface between cognition and action.

2.3 Historical Context

This distinction parallels other fundamental separations in computing history:

- **CPU vs. I/O:** Processors compute; I/O devices interact with the external world
- **Algorithm vs. Data Structure:** Algorithms process; data structures organize information
- **Control Plane vs. Data Plane:** Control makes decisions; data plane forwards packets

Similarly, the AI Cognition Layer handles thinking, the Contact Layer bridges, and the External World handles action. The contact layer serves as the structured boundary between internal reasoning and external action.

3 The Contact Layer Concept

3.1 Definition

The Contact Layer is the interface between AI intelligence and the external world.

The external world includes:

- AI capabilities (models, embeddings, generation)
- Digital services (APIs, databases, automation platforms)
- Knowledge systems (search, retrieval, knowledge bases)
- Physical devices (IoT, robotics, sensors, actuators)

The contact layer transforms internal decisions into executable actions.

Definition 1. *The Contact Layer is defined as a tuple:*

$$CL = (C, Policy, Context)$$

Where:

- *C: Set of available capabilities* $C = \{c_1, c_2, \ldots, c_n\}$
- *Policy: Function* $Policy(c, ctx) \rightarrow (provider, constraints)$
- *Context: Execution context including preferences, constraints, history*

The Contact Layer maps agent intent to capability execution:

$$Route : Intent \times Context \rightarrow Capability \times Provider \times Constraints$$

3.2 Core Functions

The contact layer performs several essential functions:

1. Capability Mediation: Translating agent intent into capability invocations. When an agent decides to "search for information," the contact layer determines which capability to invoke and with what parameters.

2. Provider Abstraction: Shielding agents from provider-specific details. Agents need not know whether search is performed by Provider A or Provider B—they express capability requirements, and the contact layer handles provider selection.

2.1 Policy Evaluation Performance: Policy evaluation can become a bottleneck under high concurrency if complex routing rules are evaluated for each request. The architecture addresses this through:

- **Synchronous vs. Asynchronous Decisions:** Simple routing decisions (based on static policies or cached preferences) are made synchronously with minimal latency. Complex decisions (requiring real-time cost optimization or policy negotiation) may be processed asynchronously, with fallback to cached decisions.

- **Policy Caching:** Frequently evaluated policy decisions are cached. For example, routing preferences for common capability-provider combinations can be cached and refreshed periodically rather than re-evaluated per-request.

- **Decision Precomputation:** For predictable workloads, routing decisions can be precomputed based on historical patterns and updated via background processes.

- **Performance Budget:** The contact layer enforces a maximum decision latency (e.g., 10ms) for synchronous routing. If policy evaluation exceeds this budget, cached or default routing is used.

3. Action Sequencing: Coordinating multiple capabilities into coherent action sequences. Complex tasks require multiple capabilities executed in specific orders, which the contact layer orchestrates.

4. Failure Policy Evaluation: Determining the appropriate response to capability failures. When a capability fails, the contact layer evaluates organizational policies to decide whether to retry, select an alternative provider, or report failure to the agent. The actual retry execution and error handling mechanics are performed by the Execution Runtime [1].

5. Context Management: Coordinating state persistence across capability invocations. Multi-step tasks require context persistence, which the contact layer orchestrates through coordination rules. The state storage and retrieval mechanics are handled by external session stores or the Execution Runtime, maintaining the contact layer's stateless design.

5.1 State Architecture: The contact layer coordinates state management but does not store state itself, preserving horizontal scalability. The architecture supports multiple state strategies:

- **Session State Store:** For multi-turn conversations, state is typically stored in a dedicated session store (e.g., Redis, in-memory cache) referenced by session ID. The contact layer reads from and writes to this store.

- **Stateless Operation:** For single-invocation scenarios, no state persistence is required. Each capability call is independent.

- **Agent-Managed State:** In some architectures, the agent maintains its own state and passes relevant context with each request. The contact layer merely transports this context.

Horizontal Scalability: To maintain horizontal scalability, the contact layer itself remains stateless—it does not maintain in-memory session state. Instead, session identifiers are passed with each request, state is persisted to external stores, and any contact layer instance can handle any request for the same session. This separation of state coordination (contact layer) from state storage (external store) enables elastic scaling while supporting multi-step workflows.

3.3 Capabilities as the Language of Interaction

Within the contact layer, the fundamental abstraction is the **capability**. A capability represents a structured action that an AI system can invoke.

Examples of capabilities include: `generate_text`, `search_web`, `send_email`, `translate_text`, `create_image`, `query_database`, and `control_device`.

Each capability defines:

- **Inputs:** Structured input parameters
- **Outputs:** Structured output format
- **Execution Semantics:** How the capability is performed

Capabilities serve as the **operational vocabulary** through which AI systems interact with the world.

4 Architectural Realization

4.1 From Concept to Architecture

While the contact layer is a conceptual model, it can be implemented through concrete system components. A practical architecture includes:

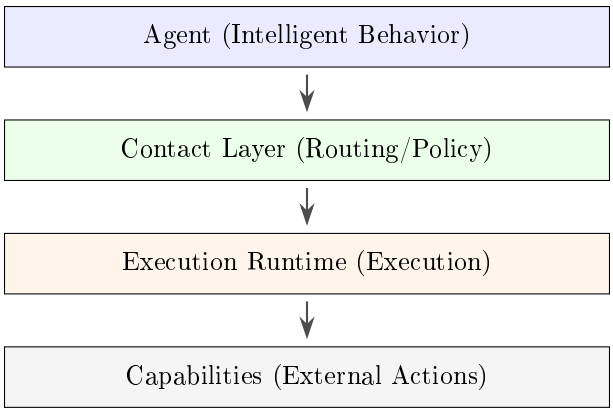

Figure 1: Contact Layer Architecture Components

Agent: Represents intelligent behavior. Agents reason about tasks, plan actions, and decide which capabilities should be invoked.

Contact Layer: Performs capability routing and application policy decisions. This component determines which capability implementation to use, which provider should be selected, and which execution environment should handle the request.

Execution Runtime: Executes capability calls. This component handles the mechanics of invoking APIs, running tools, and returning results.

Capabilities: Represent actions available to the AI system.

4.2 Clarifying Terminology Boundaries

A common source of confusion is the relationship between Contact Layer and Execution Layer. Based on architectural clarity, we establish:

Original Intuition: Many practitioners conceive of a single "execution/contact layer" that handles both routing decisions and execution mechanics. This intuition is correct—there is one macro layer that mediates between agents and capabilities.

Architectural Decomposition: For software engineering purposes, this macro layer can be decomposed into two subcomponents:

1. **Contact Layer:** Handles routing, provider selection, and policy evaluation. This is the "control plane" of the action layer.
2. **Execution Runtime:** Handles API invocation, tool execution, and result normalization. This is the "data plane" of the action layer.

Unified Understanding: The AI Action Layer (Macro Layer) comprises the Contact Layer (Routing/Policy - Control Plane) and the Execution Runtime (Execution - Data Plane). This decomposition maintains software engineering clarity while preserving the conceptual unity of the action interface.

4.3 Precise Definition

We can now state precisely:

> *The Contact Layer (in the narrow sense) is the routing and policy component of the AI Action Layer, responsible for capability routing, provider selection, and policy evaluation.*

And more broadly:

> *The Action Layer (in the broad sense) is the complete interface between AI cognition and the capability ecosystem, comprising both routing (contact) and execution (runtime) components.*

This terminology clarification resolves the apparent contradiction while preserving both conceptual clarity and architectural precision.

5 Digital and Physical Interaction

5.1 Dual Domain Support

The contact layer must support both digital and physical interactions:

Digital Interaction: Capabilities in the digital domain include:

- APIs (SaaS services, cloud platforms)
- Databases (SQL, NoSQL, vector databases)
- Cloud services (compute, storage, networking)
- Automation platforms (workflow engines, integration services)

Physical Interaction: Capabilities in the physical domain include:

- IoT devices (sensors, actuators, controllers)
- Robotics platforms (manipulators, mobile robots)
- Industrial systems (manufacturing, logistics)
- Environmental controls (HVAC, lighting, security)

5.2 The Significance of Physical Interaction

Through the contact layer, AI systems can transition from **information processing systems** to **active participants in real environments**. This transition is significant because:

- AI moves from analyzing data to affecting the world
- Systems can operate in physical spaces, not just digital ones
- New applications become possible (robotics, manufacturing, smart environments)

The contact layer provides the abstraction that enables this transition, allowing agents to express physical actions through the same capability interface used for digital operations.

5.3 Edge Computing Implications

The contact layer architecture naturally supports edge computing scenarios:

Edge Deployment: Contact layers can operate locally on:

- Personal devices (smartphones, tablets, wearables)
- Home servers (home automation, personal AI)
- Edge gateways (industrial, retail, healthcare)
- Embedded systems (automotive, aerospace, appliances)

Hybrid Operation: Edge-deployed contact layers may connect to:

- Local models (on-device AI, offline capabilities)
- Local automation systems (home automation, industrial control)
- Cloud capabilities (fallback to cloud when edge is insufficient)

This architecture enables **personal AI infrastructure**, where users operate local contact layers orchestrating personal capabilities while maintaining connection to cloud resources.

6 The Action Interface Definition

6.1 Formal Definition

We define the action interface formally:

Definition 2. *The Action Interface is the boundary between AI cognition and the capability ecosystem, defined by:*

$$\mathit{ActionInterface} = (\mathit{Capabilities}, \mathit{Routing}, \mathit{Execution})$$

Where:

- ***Capabilities:*** *The set of available capabilities* $C = \{c_1, c_2, \ldots, c_n\}$
- ***Routing:*** *A function* $\mathit{Route}(c, \mathit{context}) \to \mathit{provider}$
- ***Execution:*** *A function* $\mathit{Execute}(c, \mathit{input}, \mathit{provider}) \to \mathit{output}$

PolicyDecision Formalization: The routing decision process can be formalized as:

$$\text{PolicyDecision}(c, \text{ctx}) \to (\text{provider}, \text{constraints}, \text{metadata})$$

Where the input includes c (the capability being requested) and ctx (execution context containing preferences, constraints, and history), and the output includes the selected provider, applied constraints, and decision rationale.

Decision Logic:

$$\text{PolicyDecision}(c, \text{ctx}) = \arg \min_{p \in \text{Providers}(c)} \text{Score}(p, \text{pref}, \text{cons})$$

Where:

$$\text{Score}(p, \text{pref}, \text{cons}) = w_1 \cdot \text{LatencyScore}(p) + w_2 \cdot \text{CostScore}(p) + w_3 \cdot \text{QualityScore}(p) + \text{Penalty}(p, \text{cons})$$

with $w_1 + w_2 + w_3 = 1.0$ and $\text{Penalty}(p, \text{cons}) = 0$ if p satisfies all constraints, ∞ otherwise.

Dynamic Weight Override: The preference weights (w_1, w_2, w_3) support dynamic override by the Cognition Layer at request time. This enables agents to adjust routing behavior based on task urgency or context.

This formalization enables transparency (decisions can be explained), optimization (scoring functions can be tuned), and compliance (constraints are explicitly represented).

6.2 Precision Statement

We can state the role of the contact layer precisely:

> *The Contact Layer is the policy and routing component of the action interface between AI cognition and the capability ecosystem, responsible for capability routing, provider selection, and policy evaluation.*

This definition captures:

- **Position:** Between cognition and execution runtime
- **Function:** Routing, policy evaluation, provider selection
- **Scope:** Control plane of the Action Layer
- **Domain:** Capability mediation (not execution mechanics)

Note on Formalization: The PolicyDecision formalization above is conceptual, illustrating the decision logic framework. Detailed algorithm implementations, including constraint satisfaction, weight learning, and multi-objective optimization, are addressed in the ai-lib reference implementations.

6.3 Architectural Significance

This definition has architectural implications:

1. **Separation of Concerns:** Cognition and action are architecturally distinct, enabling independent evolution.
2. **Interface Contracts:** The action layer defines clear contracts for capability invocation, enabling provider substitution.
3. **Ecosystem Participation:** By defining the action interface, systems can participate in capability ecosystems.
4. **Decoupling Principle:** Agents need not know provider details—they express capability requirements, and the action layer handles implementation.

7 Comparison with Existing Frameworks

7.1 Current Approaches

Current AI frameworks focus primarily on agent logic and tool integration:

LangChain [3]: Provides mechanisms for tool invocation within agent workflows. However, LangChain embeds capability execution directly within the application framework, mixing orchestration and execution.

AutoGPT: Implements autonomous agents with tool use, but tightly couples planning with tool execution, limiting reusability.

Model Context Protocol (MCP) [11]: Defines capability-like tools for model interaction, but lacks a unified action layer architecture.

7.2 Distinctive Approach

The Contact Layer model differs by treating capability execution as an **independent architectural component**. This separation enables:

- **Reusable Runtimes:** Execution components can be shared across applications
- **Standardized Capability Protocols:** Capabilities can be described uniformly
- **Flexible Orchestration:** Routing strategies can evolve independently

7.3 Integration Path

Existing frameworks can integrate the contact layer concept:

1. Separate tool invocation from agent logic
2. Define capabilities through standardized protocols
3. Implement routing and execution as distinct components
4. Enable provider abstraction through capability interfaces

This integration path allows frameworks to evolve toward the contact layer architecture incrementally.

7.4 Comparison with Multi-Agent Frameworks

Table 2: Comparison of Contact Layer with Multi-Agent Frameworks

Aspect	**Contact Layer**	**OpenAI Swarm**	**CrewAI**	**LangChain**
Architecture	Layer-based separation	Agent-centric	Role-based	Tool-centric
Capability Model	Protocol-defined	Agent-owned	Task-specific	Tool-wrapped
Provider Abstraction	Full (via protocol)	Partial	Limited	Framework-specifi
Routing	Context-aware, dynamic	Handoff-based	Sequential	Chain-based
Safety Mechanisms	Layer-enforced	Agent-level	Task-level	Application-level
Runtime Portability	Protocol-standardized	Platform-locked	Framework-locked	Framework-locked
State Management	Externalized	Agent-maintained	Crew-maintained	Chain-maintained

The Contact Layer differs fundamentally by treating capability execution as an independent architectural layer rather than embedding it within agent logic or framework-specific tool abstractions.

8 The Evolution of AI Systems

8.1 Historical Progression

The introduction of the contact layer reflects a broader evolution in AI system design:

Phase 1: Model Intelligence (2017–2022)
Focus: Improving model capabilities
Examples: GPT-2, GPT-3, BERT, T5
Characteristics: Models as standalone capabilities

Phase 2: Capability Integration (2022–2025)
Focus: Connecting models to tools and services

Examples: ChatGPT plugins, LangChain, AutoGPT
Characteristics: Models with tool use
Phase 3: Environment Interaction
Focus: AI systems interacting with environments
Examples: AI-native applications, autonomous systems, edge AI
Characteristics: Action layers, capability ecosystems
Status: Current and emerging phase

This evolution can be summarized as: Intelligence → Capabilities → Contact Layer → Action.

8.2 Future Trajectory

As AI systems mature, we anticipate:

1. **Standardized Capability Protocols:** Industry-wide protocols for capability description
2. **Capability Marketplaces:** Economic ecosystems for capability provision and consumption
3. **Edge-First Architecture:** Contact layers deployed at the edge for latency-sensitive applications
4. **Physical-Digital Integration:** Seamless interaction with both digital services and physical devices

The contact layer concept provides the architectural foundation for these developments.

9 Personal AI Infrastructure

9.1 Concept

One important consequence of the contact layer concept is the emergence of **personal AI infrastructure**. A user may operate a local contact layer that connects:

- Local AI models (on-device processing, privacy-preserving)
- Personal data sources (files, emails, calendars)
- Home automation systems (smart home, IoT)
- Cloud AI capabilities (when local resources insufficient)

This creates a **unified interaction environment** in which AI systems act on behalf of users across digital and physical domains.

9.2 Architecture

9.3 Deployment Scenarios

Personal AI infrastructure can operate on:

- **Personal Devices:** Smartphones, tablets, laptops
- **Home Servers:** Dedicated hardware for home AI
- **Edge Gateways:** Industrial or commercial edge deployments
- **Embedded Systems:** Automotive, aerospace, healthcare

The contact layer architecture enables consistent capability access across these deployment scenarios.

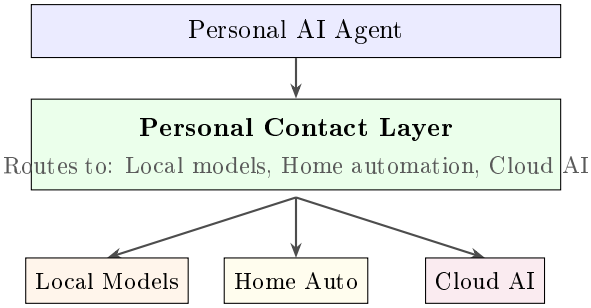

Figure 2: Personal AI Infrastructure Architecture

9.4 Personal AI Use Case: Smart Home + Email

The following sequence illustrates a personal AI scenario where the contact layer dynamically routes between local and cloud capabilities:

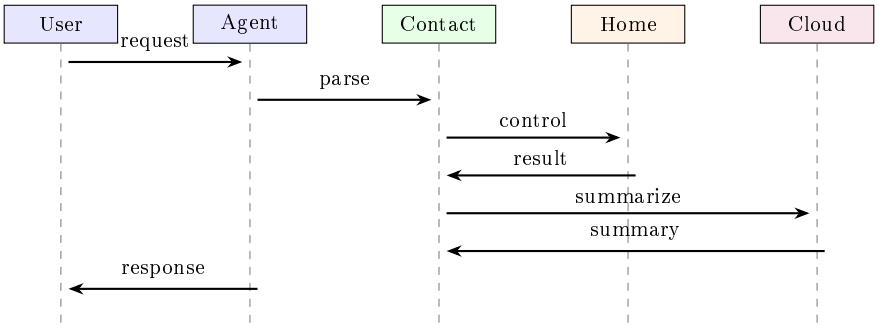

Figure 3: Personal AI Sequence: Smart Home + Email

Key Architecture Points:

1. **Local-First Routing:** Device control routes to local automation (low latency, privacy, offline-capable)
2. **Cloud Fallback:** Email summarization routes to cloud AI (more powerful, needs data access)
3. **Unified Interface:** Both requests use the same capability interface despite different execution environments
4. **Context Preservation:** The contact layer maintains request context across parallel invocations

10 Relationship to AI Native Stack

10.1 Position in the Stack

The contact layer sits within a broader AI Native Stack:

10.2 Conceptual Unity

The action layer represents the conceptual unity of the contact layer (routing) and execution runtime (execution). While architecturally decomposed, they form a single coherent layer:

Conceptual View: AI Action Layer as the interface between cognition and capabilities.

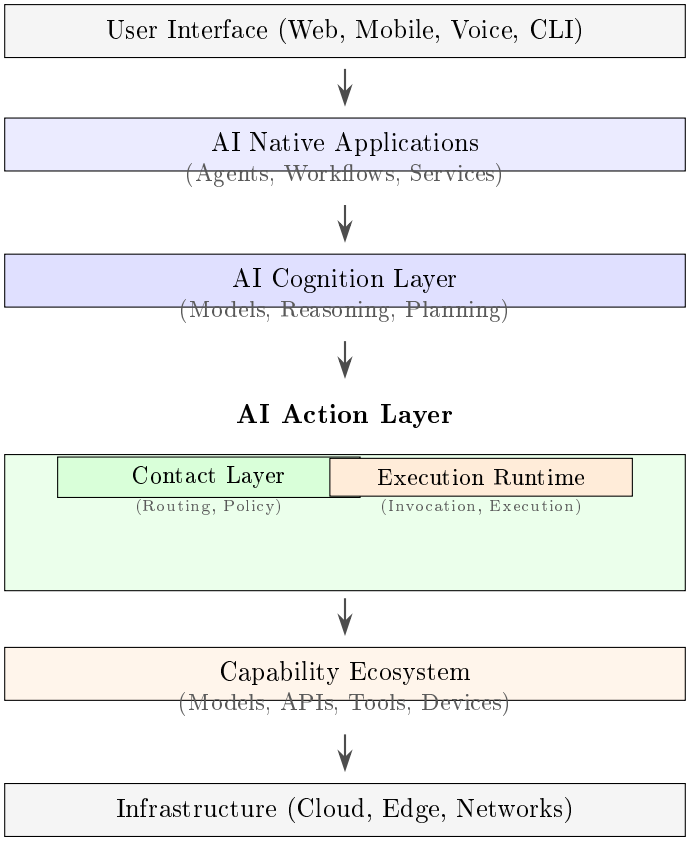

Figure 4: AI Native Stack with Contact Layer

Architectural View: Action Layer decomposed into Contact (routing) and Execution (invocation) components.

Both views are correct—they represent different levels of abstraction appropriate for different purposes.

11 Philosophical Implications

11.1 Thought and Action in AI

The contact layer concept touches on a fundamental philosophical question: What is the relationship between thought and action in AI systems?

This question echoes classic debates in AI and cognitive science, from Brooks' [3] arguments against symbolic representation to Suchman's [5] situated action theory and Dreyfus' [8] critique of computational approaches to intelligence.

Traditional AI research often focused on:

- Representation (how to encode knowledge)
- Reasoning (how to draw conclusions)
- Learning (how to improve from data)

The contact layer concept adds a new dimension:

- **Interaction:** How AI systems affect the world

This shift from "thinking about" to "acting upon" represents a fundamental transition in AI system design. These philosophical debates are not merely academic—they directly inform how we architect AI systems to interact with the world.

11.2 The Mediator Role

The contact layer serves as a **mediator** between two domains:

- **Internal Domain:** Representations, reasoning, decisions
- **External Domain:** Actions, effects, consequences

This mediation is necessary because:

1. Internal representations are not directly executable
2. External actions require specific protocols
3. Context must be maintained across the boundary
4. Failures must be handled appropriately

The contact layer provides the structured interface that enables this mediation.

11.3 Implications for AI Safety

The contact layer has implications for AI safety:

- **Action Boundaries:** Clear boundaries between cognition and action enable safety constraints at the action layer
- **Capability Limits:** The contact layer can enforce limits on which capabilities are accessible
- **Monitoring:** Action layer monitoring provides visibility into AI system behavior
- **Intervention:** Safety interventions can be implemented at the contact layer

By structuring the action interface, the contact layer provides natural points for safety mechanisms.

11.4 Safety Mechanisms at the Contact Layer

The contact layer enables several safety mechanisms through its mediation role:

Action Interception Protocol: The contact layer intercepts capability invocations to enforce safety constraints. Risk levels, confirmation requirements, and scope limits are defined in capability schemas, not hardcoded in the layer itself. This separation ensures safety policies are auditable and modifiable.

Trust Domains: Agents and users can be assigned to trust domains with different privilege levels. High-trust domains may have relaxed confirmation requirements, while restricted domains may require approval for all operations.

Batch Authorization: For trusted workflows, the contact layer supports pre-authorization of capability sequences, reducing confirmation friction while maintaining audit trails.

Pre-Flight Simulation: For high-risk physical capabilities, the contact layer can route requests through a simulation environment before execution. This is particularly important for irreversible physical actions where logical validation alone is insufficient.

These mechanisms illustrate how the Intelligence-Action separation enables safety at the architecture level, rather than embedding safety concerns within agent logic.

12 Conclusion

Intelligent behavior alone does not enable AI systems to influence the world. A structured interface between intelligence and action is required. This paper introduced the concept of the AI Contact Layer, a conceptual framework that describes how AI systems interact with external capabilities and environments.

The central claim of this paper is that the Intelligence-Action separation is not merely an architectural convenience, but a fundamental design principle for AI-native systems.

The contact layer provides:

1. **Conceptual Clarity:** A clear distinction between cognition and action
2. **Architectural Foundation:** A structured approach to implementing action interfaces
3. **Unified Perspective:** Integration of execution infrastructure with broader system design
4. **Practical Guidance:** Principles for designing capability-mediated AI systems

The contact layer serves as the bridge between internal reasoning and real-world action, enabling the emergence of AI-native systems capable of meaningful interaction with both digital and physical environments.

As AI systems evolve from information processing to environment interaction, the contact layer will play an increasingly central role in AI architecture. By defining this layer clearly, we provide the conceptual foundation for the next generation of intelligent software infrastructure.

A Capability Interaction Patterns

A.1 Simple Capability Invocation

1. Agent Decision: "Search for AI architecture papers"
2. Contact Layer: Route to `search_web` capability
3. Provider Selection: Choose Provider A (fast)
4. Execution Runtime: Invoke Provider A API
5. Result: Return search results to agent

A.2 Multi-Capability Workflow

1. Agent Decision: "Write a blog post about AI"
2. Contact Layer: Decompose into capability sequence:
   a. `search_web`("AI topics")
   b. `summarize`(search_results)
   c. `generate_text`(blog_outline)
   d. `edit_text`(generated_draft)
3. Execution: Execute each capability in sequence
4. Coordination: Pass results between capabilities
5. Result: Return final blog post

A.3 Error Handling Pattern

1. Agent Request: invoke capability X
2. Contact Layer: Route to Provider A
3. Execution: Provider A fails (timeout)
4. Contact Layer: Retry with Provider B
5. Execution: Provider B succeeds
6. Result: Return result from Provider B

B Capability Definition Schema

B.1 Capability Schema

```yaml
capability_schema:
  name: string
  version: string
  description: string
  input_schema:
    type: object
    properties: {}
    required: []
  output_schema:
    type: object
    properties: {}
  execution:
    endpoint: string
    method: enum [GET, POST, PUT, DELETE]
    timeout_ms: integer
    retry_policy:
      max_retries: integer
      backoff_ms: integer
  provider:
    name: string
    version: string
  metadata:
    category: enum [AI, API, Tool, Device]
    tags: [string]
    cost_per_call: number
```

B.2 Example Capability

```yaml
name: control_device
version: "1.0"
description: Control an IoT device
input_schema:
  type: object
  properties:
    device_id:
      type: string
    action:
      type: string
      enum: [turn_on, turn_off, set_level]
```

```yaml
      level:
        type: number
        description: Level for set_level action
    required: [device_id, action]
  output_schema:
    type: object
    properties:
      status:
        type: string
        enum: [success, failure]
      device_state:
        type: object
      timestamp:
        type: string
  execution:
    endpoint: /v1/devices/{device_id}/control
    method: POST
    timeout_ms: 5000
    retry_policy:
      max_retries: 2
      backoff_ms: 1000
  provider:
    name: iot_controller
    version: "2.1"
  metadata:
    category: Device
    tags: [iot, physical, automation]
    cost_per_call: 0.001
```

C Contact Layer Interface Specification

C.1 Routing Interface

```typescript
interface ContactLayer {
  /**
   * Route a capability request to an appropriate provider.
   */
  route(request: CapabilityRequest): Promise<RoutingDecision>;

  /**
   * Evaluate policies for capability invocation.
   */
  evaluatePolicy(
    capability: string,
    context: ExecutionContext
  ): Promise<PolicyDecision>;

  /**
   * Select provider based on context.
   */
  selectProvider(
    capability: string,
    context: ExecutionContext
  ): Promise<Provider>;

  /**
```

```
    * Choose execution environment.
    */
  selectEnvironment(
    capability: string,
    provider: Provider
  ): Promise<Environment>;
}

interface RoutingDecision {
  capability: string;
  provider: Provider;
  environment: Environment;
  policyCompliant: boolean;
  estimatedLatency: number;
  estimatedCost: number;
}
```

C.2 Capability Request Structure

```
interface CapabilityRequest {
  // Capability identification
  capability: string;
  version?: string;

  // Input data
  input: Record<string, any>;

  // Execution context
  context: {
    sessionId: string;
    requestId: string;
    userId?: string;
    priority?: 'low' | 'medium' | 'high';
    deadline?: Date;
  };

  // Routing preferences
  preferences?: {
    preferredProvider?: string;
    maxLatency?: number;
    maxCost?: number;
    requirePrivacy?: boolean;
  };
}
```

References

[1] Wang, L. (2026). The AI Execution Layer: A Minimal Runtime Architecture for AI-Native Systems. ai-lib Preprint v1.5.

[2] Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

[3] Brooks, R. A. (1991). Intelligence Without Representation. Artificial Intelligence, 47(1-3), 139-159.

[4] Agre, P. E., & Chapman, D. (1987). Pengi: An Implementation of a Theory of Activity. AAAI.

[5] Suchman, L. A. (1987). Plans and Situated Actions. Cambridge University Press.

[6] Gibson, J. J. (1979). The Ecological Approach to Visual Perception. Houghton Mifflin.

[7] Winograd, T., & Flores, F. (1986). Understanding Computers and Cognition. Addison-Wesley.

[8] Dreyfus, H. L. (1972). What Computers Can't Do. MIT Press.

[9] Searle, J. R. (1980). Minds, Brains, and Programs. Behavioral and Brain Sciences, 3(3), 417-424.

[10] Clark, A. (1997). Being There: Putting Brain, Body, and World Together Again. MIT Press.

[11] OpenAI. (2023). GPT-4 Technical Report. arXiv:2303.08774.

[12] Anthropic. (2024). Claude Model Card. `https://www.anthropic.com/claude`

Document Information

Document Type:	Preprint
Version:	1.5
Last Updated:	March 30, 2026
Classification:	Theoretical Architecture Paper
Review Status:	Major Revision Complete
Series:	AI Native Architecture (Paper 2 of 5)

Acknowledgments

The authors thank early readers and contributors to the ai-lib and AI Protocol implementations for their feedback.

Open Source Resources: Reference implementations are available at `https://github.com/ailib-official`, including ai-lib-rust, ai-lib-python, ai-lib-ts, ai-lib-go, and ai-protocol. The AI Protocol specification is maintained at `https://github.com/ailib-official/ai-protocol`.